

Hostel Room Swapping Service

Project Proposal for UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer

Thapar Institute of Engineering and Technology

Aabhas Khandelwal, CSED* Aakash Chawla, CSED[†] Joy Soni, CSED[‡]

August 24, 2026

1 Higher-order goal

... secondary framing

Make better use of hostel rooms that already exist. Every semester students live in rooms that suit them poorly while, in the same hostel, someone would gladly take that room and give up theirs. Fixing this needs no construction — only visibility and coordination.

2 Time-to-value

... primary heuristic

- The matching engine was built and demonstrated end to end on synthetic data before any database existed, so the core proposition was validated in the first iteration.
- The domain layer has no framework or database dependency, so the full test suite runs in about two seconds on every push and a working demonstration is always available.
- Success is measured, not asserted: the gain in cohort preference satisfaction per matching round.

3 Problem Statement

Students are allotted rooms they did not choose — wrong floor, wrong direction, away from friends. They are willing to move. The obstacle is visibility, not willingness.

- **Pairwise swaps rarely work.** A direct exchange needs two students each to want exactly what the other holds. A three- or four-way rotation usually exists, but no participant can see it from inside.
- **No directory of intent.** Nothing records who wants to move or where, so willingness stays invisible.
- **Informal coordination does not scale.** A group chat surfaces a few offers; it cannot search a hostel for a closed exchange cycle.
- **No memory.** The same negotiation restarts every semester.

Contextual relevance: Hostel A alone has eight floors of roughly 58 rooms. Possible exchanges grow far faster than any student or warden can enumerate, and the cost of missing them is borne by students for a full academic year.

*Roll No: 1024030673, Email: akhandelwal2.be24thapar.edu

[†]Roll No: 1024031097, Email: achawla1.be24thapar.edu

[‡]Roll No: 1024030674, Email: jsoni.be24thapar.edu

4 Proposed Solution

... *Software Proposition*

A web service that holds every current allocation and every stated preference, and computes the exchange cycles hidden inside them.

1. A student names one specific room. If it cannot be obtained, the request falls back to that room's floor, then its direction.
2. The service builds a students $\times$ rooms cost matrix and solves for the allocation of maximum total satisfaction.
3. That allocation is decomposed into independent exchange cycles; any cycle leaving a participant worse off is discarded.
4. Each surviving cycle is issued as a proposal. All members approve, or it lapses.
5. On full approval the exchange executes and new allocation rows are appended — history is never rewritten.

Each participant is shown their new room, the resulting match percentage, and exactly which preferences were met and which were not.

5 Solution Approach

... *Engineering focus*

We assemble mature engines rather than invent algorithms. The engineering contribution is the modelling, and the feasibility layer separating allocations that are mathematically optimal from those students would actually accept.

5.1 Key modelling observation

The pool is *closed*: the only rooms available are those the participants already hold. So any solution is a **permutation** of students over rooms, and every permutation decomposes uniquely into disjoint cycles. The exchange chains are therefore not searched for separately — they are already inside the allocation and are simply read out. A 2-cycle is a direct swap; a 3-cycle is a three-way rotation. Because cycles are disjoint, each executes independently, so a participant withdrawing from one provably cannot affect any other.

5.2 Four modules

... *mature engines*

1. **Preference scoring.** Match percentage is the fraction of a student's preference weight a room satisfies. Hard constraints rule a room out rather than lowering its score.
2. **Assignment optimisation.** The **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`) solves the matrix in $O(n^3)$.
3. **Alternative generation.** **Murty's algorithm** yields the K best allocations by forbidding an edge of the best and re-solving. This is a correctness requirement: the single optimal allocation is often unexecutable, and the alternatives are where a workable exchange is found.
4. **Swap feasibility.** Cycle decomposition plus a Pareto filter, discarding any cycle where a participant ends up worse off than today.

5.3 Alternatives rejected

Gale–Shapley assumes two-sided matching where both sides rank each other; rooms hold no preferences over students, so half the algorithm has no input. **Top Trading Cycles** fits the exchange shape but operates on a strict ranked list, and cannot represent preferences weighted across several attributes.

5.4 Architecture and CI/CD

A **React** frontend over a stateless **FastAPI** backend and managed **PostgreSQL** (Supabase, nine tables). The allocation table is append-only, enforced by a database trigger rather than application convention. The domain layer imports no framework and no database driver, so it is testable with no infrastructure at all. GitHub Actions runs the full suite on every push across Python 3.10–3.12 and republishes the documentation on merge.

6 Evaluation Criterion

... *measurable and attributable*

6.1 Primary metric

Cohort preference satisfaction: the mean weighted preference score across *all* participants — not only those who move — before and after a matching round. Both figures come from the same scoring function over the same stored preferences, so the difference is attributable entirely to the reallocation.

On a 30-student cohort in hostel A, satisfaction rose from **11%** to **77%**.

Measuring across everyone is deliberate: averaging over movers alone would reward a two-person swap where both reach 100% over a 28-person rotation that lifts the whole hostel.

6.2 Secondary metrics

Metric	Result
Received the exact room they named	20 of 30
Received at least their requested floor	28 of 30
Received at least their requested direction	23 of 30
Participants left worse off	0 of 30
Longest chain needing simultaneous approval	11

Nobody worse off is a guarantee, not an observation: a cycle is only offered if every member is at least as well off as before and one is strictly better. **Chain length** matters operationally — an exchange needing 28 simultaneous approvals is far more fragile than three of 11, 10 and 7. Raising K cut the longest chain from 28 to 11 *at identical satisfaction*, which is precisely what Murty’s algorithm contributes.

6.3 Pilot validation

Collect real preferences from 20–40 volunteers in one hostel, run a matching round, and report before/after satisfaction against an honest baseline: how many direct pairwise swaps those same students could arrange informally.

7 Scalability

...with foresight

1. **Theoretically:** Hungarian is $O(n^3)$ and Murty's adds a bounded number of further solves. A 30-student pool matches in under 100 ms, a 100-student pool in under a second. Matching also partitions naturally — each hostel solved independently — so problem size stays bounded as campus participation grows.
2. **Operationally deployable:** the API is stateless over managed PostgreSQL, so it scales horizontally behind a load balancer on a standard platform host, with CI/CD producing repeatable deployments.

8 Engine availability heuristic

Every component exists as a mature engine: **SciPy** (Hungarian), **NumPy** (cost matrix), **React** (frontend), **FastAPI** (REST API), **Supabase** (managed PostgreSQL and auth), **pytest** and **GitHub Actions** (tests and CI). Murty's algorithm is about sixty lines on top of the SciPy solver and introduces no new mathematics — it repeatedly forbids one assignment edge and re-solves. Nothing here depends on unpublished or unproven work.

9 Project scope and deliverables

...iteration-friendly

9.1 Initial deliverable

- Weighted preference model with ordered fallbacks: room, then floor, then direction.
- Matching engine: cost matrix, Hungarian, Murty's K -best, cycle decomposition, Pareto filter.
- Approval state machine: proposal, approval, execution, rejection, expiry, withdrawal.
- Web view showing the allocation table, the computed match, and the cycles being executed.
- Schema with append-only allocation history.
- CI running the suite on three Python versions on every push.

9.2 Subsequent deliverables

- React frontend with student login and real preference collection, replacing the seeded demonstration cohort.
- Persisted proposals and approvals surviving restart.
- Shared rooms, with mutual roommate requests merged before solving.
- Warden view for oversight and for committing an approved round.

10 Risks and mitigations

- **Participation:** cycles need enough participants. Run per hostel, where students already coordinate, and report honestly when no exchange exists.
- **Withdrawal mid-approval:** mitigated structurally — cycles are disjoint, so a withdrawal cancels one and cannot touch the others; preferring several short cycles over one long one shrinks the blast radius further.

- **Gaming:** hard constraints only remove infeasible rooms and earn no score, so exaggerating them cannot improve a match.
- **Administrative acceptance:** exchanges remain proposals until approved, and the append-only history is a complete, non-rewritable audit trail.
- **Privacy:** preferences and allocations only; nothing beyond what matching requires.

11 Summary

This project proposes a deployable web service that finds multi-party room exchanges no participant could discover alone. It meets the course criteria through time-to-value — a working, measured engine in the first iteration — CI/CD that keeps feedback at seconds rather than days, and a measurable evaluation criterion in cohort preference satisfaction, which rose from 11% to 77% with no participant left worse off. It uses only mature engines; the engineering contribution is modelling the hostel as a closed permutation problem and the feasibility layer that separates optimal allocations from acceptable ones.